M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2026-2027)

TP1

Optimisation des accès mémoire

Utilisation de la mémoire partagée

adrien.roussel@cea.fr

L'objectif de ce TP est de comprendre les différentes mémoires existantes d'un GPU et leur utilisation sur un noyau de calcul classique d'algèbre linéaire : le produit matriciel. Nous allons voir à travers cet exemple comment utiliser la mémoire globale, la mémoire partagée, la mémoire constante et la mémoire unifiée.

Les objectifs de ce TP sont :

— Maitriser la chaine de compilation CUDA

— Compréhension des accès mémoires sur le GPU,

— Optimisation des performances avec l'utilisation de la mémoire partagée.

Le code source principal est la continuité de votre code réalisé lors de la séance précédente.

I Chaîne de compilation

Avant toute chose, pensez bien à charger le module cuda sur la machine ruche, via la commande : `module load` (vous pouvez aussi utiliser la commande `module spider` pour avoir plus de détail sur un module installé). Pour observer certaines étapes du processus de compilation, compiler le fichier *tp1_check.cu* avec la commande :

nvcc –keep –keep-dir TMP -c tp1_check.cu

(il faudra avoir créé le répertoire `CUDAIMG` avant de lancer la commande).

Des fichiers intermédiaires sont créés dans le répertoire CUDAIMG.

Q.1: Quel fichier correspond au PTX ? Lequel correspond au binaire CUDA ?

Q.2: Quelles parties des sources de *tp1_check.cu* ont été conservées pour être compilées en PTX ?

Q.3: À quoi correspond le suffixe ***__30*** des fichiers créés ?

II Multiplication matricielle

Le but de cette partie est de vous faire écrire un premier programme avec CUDA. La mémoire globale est accessible depuis n'importe quel thread d'une grille sur un GPU. C'est la mémoire qui est allouée/libérée grâce aux fonctions `cudaMalloc` et `cudaFree`. Nous allons dans un premier temps utiliser cette mémoire pour écrire un premier kernel de produit matriciel.

La multplication de deux matrices est un problème qui se prête bien au système de fonctionnement du GPU. Pour vous aider, un code séquentiel écrit en C est à votre disposition, il s'agit du fichier `dgemm.c`. La fonction *dgemm* calcule l'opération suivante : $C = \alpha AB + \beta C$
A partir d'ici, veillez à compléter ce programme afin de déporter les calculs de la multiplication des deux matrices sur un GPU. N'oubliez pas de changer l'extension du fichier (*i.e.* `.c` en `.cu`).

Soient A, B et C trois matrices carrées de tailles $N \times N$, α et β deux scalaires, la multiplication de la matrice A par la matrice B stockée dans C peut se définir par l'algorithme suivant :

```
1  double A[N*N];
2  double B[N*N];
3  double C[N*N];
4
5  double alpha, beta;
6
7  for (int i = 0; i < N; i++)
8  {
9    for (int j = 0; j < N; j++)
10   {
11     float tmp = 0.0f
12     for (int k = 0; k < N; k++)
13     {
14       tmp += A[i * N + k] * B[k * N + j];
15     }
16     C[i * N + j] = alpha * tmp + beta * C[i * N + j];
17   }
18 }
```

1 Allocation et transfert de données

Q.4: Dans un premier temps, à l'aide de la fonction `cudaMalloc`, allouer trois vecteurs de tailles $N \times N$ sur le GPU : deux vecteurs en entrée pour les opérandes de la multiplication, et un troisième pour stocker le résultat final.
Ici, nous représentons une matrice sous la forme d'un seul vecteur de taille $N \times N$ pour avoir une matrice stockée en mémoire contigue, et ne pas avoir à faire des transferts de tableaux à outrance (par exemple 1 tableau par ligne si l'on stocke une matrice sous forme d'un vecteur de vecteurs). Pensez également à libérer votre mémoire en fin de programme avec `cudaFree`.

Q.5: Les données étant maintenant allouées, transférez les données initialisées par l'hôte (*i.e.* `Host`) vers l'accélérateur (*i.e.* `Device`) en utilisant la fonction `cudaMemcpy`.

2 Configuration du kernel

Q.6: Avant de déporter du calcul sur l'accélérateur, il faut configurer la grille sur laquelle vont s'exécuter l'ensemble des threads CUDA. À l'aide des structures

de type `dim3`, configurez une grille contenant des blocs 2D de taille 32x32 (*i.e.* 1024 threads par bloc). Cette grille devra contenir suffisamment de threads afin que chacun d'entre eux puisse être en charge du calcul d'un élément de la matrice résultat. Pour plus de facilités, prenez une taille de matrice N qui soit multiple de 32.

Q.7: L'heure est venue de faire s'exécuter le premier kernel sur un accélérateur de calcul. Écrire un premier kernel, pour l'instant vierge, sur la grille préalablement configurée à la question précédente.

Q.8: En l'état, le noyau s'exécute sur l'accélérateur de calcul mais ne retourne aucun résultat sur la partie hôte. A la suite de l'exécution du kernel, veiller à transférer le résultat qui nous intéresse pour le calcul de la multiplication matricielle.

3 Écriture du kernel

Q.9: Dans le noyau de calcul, à partir des dimensions et des indices relatifs à la grille, calculez l'indice de la ligne et de la colonne de l'élément de C qu'il faudra manipuler.

Q.10: Pour un thread donné, effectuez le calcul de la case de C dans le noyau exécuté par le GPU.

4 Analyse des performances

Q.11: Maintenant que le calcul de multiplication matricielle fonctionne, il nous faut analyser les performances de notre application. Le SDK CUDA fournit des fonctions permettant de calculer le temps total passé dans un noyau de calcul. Utiliser ces fonctions et regardez le temps mis par l'application pour calculer le produit de deux matrices et le transfert du résultat sur la partie hôte.
Comparez ces performances au noyau s'exécutant uniquement sur CPU, de manière séquentiel.
Indice : la fonction à utiliser pour calculer le temps écoulé entre un point A et un point B est la fonction `cudaEventElapsedTime`.

Q.12: Quelle est la complexité arithmétique de cet algorithme ? Calculez le nombre d'opérations par secondes (FLOP/s) de votre implémentation.